

SYSTEM DESIGN MINI PROJECT

API Gateway — System Documentation

Problem Statement 26: API Gateway (Routing + Authentication + Rate Limiting)

Theme:

Microservices Architecture & Backend Infrastructure

1. Introduction

The API Gateway is designed as a centralized entry point for all client requests in a microservices-based system. Instead of clients directly communicating with multiple backend services, every request first passes through the gateway.

The gateway is responsible for handling:

- Authentication
- Authorization
- Rate Limiting
- Request Validation
- Request Routing
- Logging and Monitoring
- Fault Tolerance

This architecture improves security, scalability, maintainability, and simplifies communication between frontend applications and backend microservices.

2. Project Objective

The main objective of this project is to design and implement an API Gateway similar to systems such as:

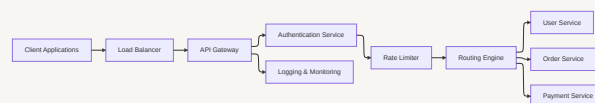
- Kong
- Amazon Web Services API Gateway

The system should:

- Route incoming requests dynamically
- Authenticate users using JWT/API Keys
- Apply rate limiting
- Validate incoming requests
- Protect backend services from abuse
- Provide centralized monitoring and control

3. System Architecture

Architecture Diagram



4. High-Level Design (HLD)

4.1 Main Components

Client Layer

The client layer includes:

- Web applications
- Mobile applications

- Third-party systems

Clients communicate with the gateway using HTTP/HTTPS requests.

Load Balancer

The load balancer distributes incoming traffic across multiple API Gateway instances.

Responsibilities:

- Prevent server overload
- Improve scalability
- Increase availability
- Remove single points of failure

Examples:

- Nginx
 - HAProxy
 - AWS ALB
-

API Gateway

The API Gateway acts as the main control layer.

Responsibilities:

- Authentication
 - Authorization
 - Rate limiting
 - Routing
 - Request transformation
 - Logging
 - Error handling
-

Authentication Service

Responsible for:

- JWT token generation
 - Token validation
 - API key verification
 - User role management
-

Rate Limiter

Controls how many requests a user/IP can make within a time window.

Purpose:

- Prevent abuse
 - Prevent DDoS attacks
 - Ensure fair usage
-

Routing Engine

Determines which backend service should receive the request.

Example:

Request Path	Service
/api/users	User Service
/api/orders	Order Service
/api/payments	Payment Service

Backend Services

The project contains independent microservices:

- User Service
- Order Service

- Payment Service

Each service handles its own business logic.

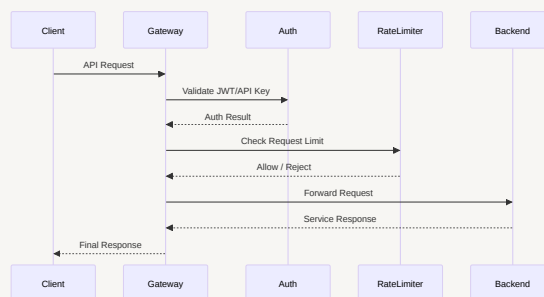
Logging & Monitoring

Tracks:

- Request count
- Response time
- Errors
- User activity
- Traffic metrics

5. Data Flow

Request Lifecycle



6. Middleware Pipeline

Requests pass through multiple middleware layers in sequence.

```

Request
↓
Logger Middleware
↓
Authentication Middleware
  
```

```
↓  
Rate Limiter Middleware  
↓  
Request Validator  
↓  
Request Transformer  
↓  
Circuit Breaker  
↓  
Routing Engine  
↓  
Backend Service  
↓  
Response
```

7. Authentication Design

Supported Authentication Methods

JWT Authentication

Clients send tokens using:

```
Authorization: Bearer <token>
```

API Key Authentication

```
X-API-Key: <api_key>
```

JWT Structure

```
{  
  "id": "user_id",  
  "username": "john",  
  "role": "admin",  
  "iat": 123456789,  
}
```

```
"exp": 123456999
}
```

Authentication Workflow

1. Extract token from request headers
2. Verify token signature
3. Check expiration time
4. Validate permissions
5. Allow or reject request

8. Authorization (RBAC)

Role-Based Access Control determines what users can access.

Role	Access
Admin	All services
User	Limited services
Anonymous	Public endpoints only

9. Rate Limiting Design

Sliding Window Algorithm

The system tracks requests within a fixed time window.

Example:

- Window: 15 minutes
- User limit: 100 requests

If limit exceeds:

```
{  
  "error": "Too many requests"  
}
```

Rate Limit Headers

```
X-RateLimit-Limit: 100  
X-RateLimit-Remaining: 87  
X-RateLimit-Reset: 1740000000
```

10. Circuit Breaker Design

The circuit breaker prevents cascading failures when backend services become unavailable.

States

CLOSED

Requests are forwarded normally.

OPEN

Requests are blocked immediately.

HALF_OPEN

Allows limited test requests to check recovery.

Benefits

- Prevents resource exhaustion
- Improves fault tolerance
- Prevents cascading failures

11. API Endpoints

Authentication APIs

Method	Endpoint	Description
POST	<code>/auth/register</code>	Register user
POST	<code>/auth/login</code>	Generate JWT
POST	<code>/auth/refresh</code>	Refresh token

Gateway APIs

Method	Endpoint	Description
GET	<code>/health</code>	System health
GET	<code>/gateway/metrics</code>	Metrics
GET	<code>/gateway/logs</code>	Request logs

Proxied Service APIs

Gateway Route	Backend Service
<code>/api/users/*</code>	User Service
<code>/api/orders/*</code>	Order Service
<code>/api/payments/*</code>	Payment Service

12. Database and Cache Design

In-Memory Stores

Rate Limit Store

Stores request counters per user/IP.

Auth Store

Stores:

- Users
- Password hashes
- API keys
- Roles

Circuit Breaker Store

Stores service health states.

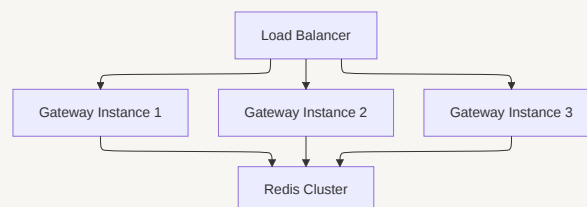
Production Upgrade

In production, replace in-memory storage with:

- Redis
 - PostgreSQL
 - MongoDB
-

13. Scalability Design

Horizontal Scaling



Scalability Techniques

- Multiple gateway instances
- Load balancing

- Distributed rate limiting
 - Token caching
 - Async logging
-

14. Security Design

Security Layers

Transport Security

HTTPS/TLS encryption

Authentication

JWT/API Key validation

Authorization

Role-based access control

Rate Limiting

Protection against abuse

Input Validation

Request payload validation

Header Security

Sensitive header filtering

15. Project Structure

```
api-gateway/  
|  
├─ gateway/  
|   └─ middleware/
```

```
| | | services/
| | | config/
| | | server.js
| |
| | microservices/
| | | userService.js
| | | orderService.js
| | | paymentService.js
| |
| | dashboard/
| |
| | docs/
| |
| | README.md
```

16. Tech Stack

Layer	Technology
Runtime	Node.js
Framework	Express.js
Proxy	http-proxy-middleware
Authentication	JWT
Password Hashing	bcryptjs
Cache	Redis
Monitoring	Dashboard + Metrics
Deployment	Docker

17. Advantages of API Gateway

Centralized Security

Authentication and authorization handled in one place.

Simplified Client Communication

Clients only communicate with one endpoint.

Better Monitoring

Centralized logging and analytics.

Traffic Control

Rate limiting and throttling.

Improved Scalability

Supports horizontal scaling.

18. Trade-Offs

Advantages

- Centralized management
- Better security
- Simplified routing
- Easier monitoring

Disadvantages

- Potential bottleneck
 - Single point of failure if not scaled
 - Additional infrastructure complexity
-

19. Future Improvements

Possible future upgrades include:

- Kubernetes deployment
- Redis clustering
- OAuth2 support

- WebSocket support
 - GraphQL gateway
 - AI-based anomaly detection
 - Service discovery integration
-

20. Conclusion

The API Gateway provides a secure, scalable, and centralized solution for managing communication between clients and microservices.

The project demonstrates:

- Request routing
- Authentication
- Authorization
- Rate limiting
- Fault tolerance
- Monitoring
- Scalability concepts

This architecture reflects real-world backend infrastructure patterns used in modern cloud-native systems and enterprise microservices environments.